Software Architecture Specification

for

AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices

Version 1.1 approved

Prepared by

Victor Gan, Tobias Jun, Atticus Kong

Mark Pham, Jacob Sepkus, Nick Williamson

24 October, 2025

Table of Contents

1. Introduction.....5
1.1 Purpose.....5
1.2 Scope.....5
1.3 Terminology.....5
1.3.1 Acronyms.....5
2. Architecture Overview.....6
2.1 Identification.....6
2.2 Goals and Objectives.....6
2.3 Architectural Model.....6
2.4 System Context Diagram.....7
3. On-Device Architectural Layer – Presentation.....8
3.1 User Interface.....8
3.1.1 Introduction.....8
3.1.2 Structure.....8
3.1.3 Flow of Operation.....8
3.1.4 Interfaces.....8
3.1.5 Risks and Mitigations.....8
3.1.6 Acceptance and Demonstration Criteria.....8
3.2 Alerts.....9
3.2.1 Introduction.....9
3.2.2 Structure.....9
3.2.3 Flow of Operation.....10
3.2.4 Interfaces.....10
3.2.5 Risks and Mitigations.....10
3.2.6 Acceptance and Demonstration Criteria.....10
4. On-Device Architectural Layer – Data.....12
4.1 Data Collection.....12
4.1.1 Introduction.....12
4.1.4 Performance and Scheduling.....12
4.1.5 Pre-Processing and Feature Windows.....12
4.1.6 Data Contracts.....12
4.1.7 Error Paths and Fallbacks.....12
4.1.8 External Interfaces.....13
4.1.9 Acceptance and Demo Criteria.....13
4.1.10 Risks and Mitigation.....13
4.1.11 Traceability.....13
4.2 Permissions Policy.....13
4.2.1 Introduction.....13
4.2.2 Required Access and Justification.....13

4.2.3 Permission UX Flows....14
4.2.4 Privacy and Data Handling....14
4.2.5 OS Constraints....14
4.2.6 Risks and Mitigations....14
4.2.7 Logging and Auditing....14
4.2.8 Acceptance and Demo Criteria....14
4.2.9 Traceability....15
4.3 Local Storage....15
4.3.1 Introduction....15
4.3.2 Interfaces....15
4.3.3 Operation....15
4.3.4 Relationship to Backend....15
4.3.5 Model Handling in Local Storage....16
4.3.6 Security and Privacy....16
4.3.7 Error Handling....16
4.3.8 Performance....16
4.4 Feature Extraction....16
4.4.1 Overview....16
4.4.2 Structural Details....17
2. Feature Extractor (Core Transformer)....17
3. Feature Repository (Producer of Outputs)....18
4.4.3 Interfaces....18
4.4.3 Sequence Diagram....19
4.5 Model Runner....20
4.5.1 Overview....20
4.5.2 Structural Details....20
4.5.3 Interfaces....21
4.5.4 Sequence Diagram....23
5. Off-Device Architectural Layer – Online Storage....24
5.1 Introduction....24
Figure 5-1 – System Context (Online Storage Layer)....24
5.2 System Structural Overview....24
5.2.1 Software Component View....25
5.2.2 Data Organisation and Access Paths....25
5.3 Theory of Operation....26
5.3.1 Config fetch (live knobs)....26
5.3.2 Model discovery, download, and activation....26
5.3.3 Telemetry ingest....27
5.4 List of External Interfaces....28
5.5 Security and Access Control....29
5.6 Product Deliverables (Storage)....29

5.6.1 Programmatic Interfaces (APIs)....29
5.7 Debugging, Error Logging, and Performance Monitoring.... 30
5.7.1 Observability & Logging....30
5.7.2 Metrics & Dashboards....30
5.7.3 Troubleshooting & Retention.... 30
6. Off-Device Architectural Layer – Offline ML Training.... 31
6.1 Introduction....31
6.2 Structure Explanation.... 31
6.2.1 View of Component....31
6.2.2 Relationship to Local Storage Component.... 32
6.2.2.1 On-Device Model Integration.... 32
6.2.2.2 ONNX ai.onnxruntime Interface.... 32
6.2.2.2.1 What It Does.... 32
6.2.2.2.2 ai.onnxruntime.OrtEnvironment.... 32
6.2.2.2.3 ai.onnxruntime.OrtSession....32
6.3 How It Works....32
6.3.1 Feature Preprocessing....32
6.3.2 Model Training.... 33
6.3.3 ONNX....33
6.4 Performance Monitoring.... 33
7. References.... 33
8. Group Attribution....36

1. Introduction

1.1 Purpose

This document explains the architecture of the program made for the AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices project, breaking down the major components and their relationships and interactions between each other. This document serves as a bridge between the requirements defined in the Software Requirements Specification (SRS) and the detailed design/implementation tasks carried out in subsequent phases.

1.2 Scope

The program is a security solution for Android mobile devices that detects and mitigates data-exfiltration malware through app activity on Android devices.

The solution is divided into three key layers on-device: the Presentation, Data and Online Storage layers, and one key layer off-device: the ML Training layer. These layers include major components that interact with each other to provide security.

1.3 Terminology

1.3.1 Acronyms

Acronym	**Phrase**
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CI	Continuous Integration
CPU	Central Processing Unit
DAO	Data Access Object
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning

ONNX	Open Neural Network Exchange
OS	Operating System
sha256	Secure Hash Algorithm 256-bit
SRS	Software Requirements Specification
TIS	Technology Investigation Summary
TTL	Time to Live
UI	User Interface

2. Architecture Overview

2.1 Identification

This document describes the software architecture of the AI-Driven Detection of Data-Exfiltration Malware on Mobile Devices system. The architecture reflects decisions made for the TIS and formalised requirements in the SRS. It covers high-level design principles, and software components and their interactions.

2.2 Goals and Objectives

The primary goals of this document are to ensure accuracy in classifying between benign and malicious apps, efficiency on Android mobile devices which have limited resources, and compliance with security and privacy requirements.

The objectives are to document component responsibilities as well as the relationships between these components, and define interfaces and data flows. This specification also establishes expectations for performance, reliability, and security. This document serves as a reference for relevant parties on the architectural decisions for the program prior to detailed design and implementation.

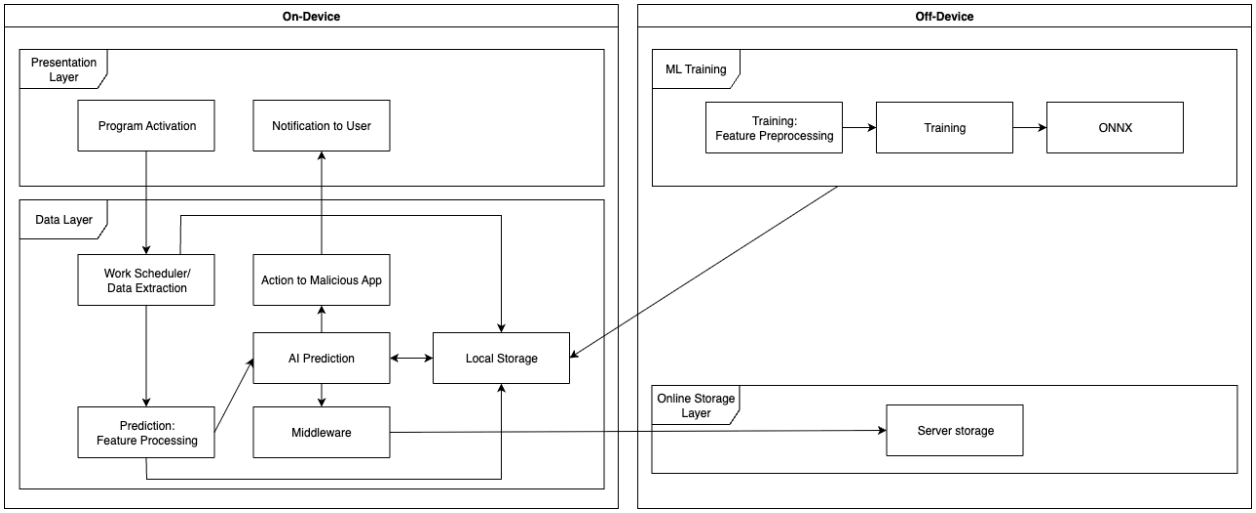
2.3 Architectural Model

Layer: On-Device	**Description**
Presentation	Includes all components that have to do with UI (what the user sees and can interact with)

Data	Includes all components that provides the function of the app
Layer: Off-Device	
Online Storage	Includes the component that is related to external storage
Offline ML Training	Includes the component that is related to training the AI model that will be integrated into the software

2.4 System Context Diagram

This system functions as a mobile-first security application that collects data from the device, processes it into features, and applies on-device prediction to detect suspicious activity. Results are shown to the user through a lightweight interface, while data may also be stored locally or synchronised with external services. Off-device, the system supports model training and storage, allowing updates to be distributed back to devices. Authentication and notifications are handled through standard platform services. Overall, the diagram illustrates the relationship between on-device layers (presentation, data, prediction, storage) and off-device components (training, storage), framed within the broader system context.



3. On-Device Architectural Layer – Presentation

3.1 User Interface

3.1.1 Introduction

The user interface is the primary interaction point between the user and the detection system. It is implemented in Kotlin with Jetpack Compose and styled with Material 3, which provides a modern, responsive, and accessible design system for Android development [17].

3.1.2 Structure

The UI comprises three main screens: a dashboard showing device health and recent detections, a history view that allows browsing of past alerts and logs, and a settings screen where users can configure preferences such as notification behaviour and data synchronisation. Navigation between screens is handled using Navigation Compose, which manages back-stack consistency and state preservation [18].

3.1.3 Flow of Operation

When the application launches, the dashboard retrieves the most recent status from local storage and subscribes to state updates from collectors and the ONNX Runtime model runner. Suspicious behaviour detected by the model is reflected immediately on the dashboard, and users can drill into the history view for details. The settings screen is accessed through the navigation graph and provides persistent configuration options.

3.1.4 Interfaces

The UI communicates with repositories that abstract Room for structured data and internal storage for model files. It consumes output from the model runner to present alerts and summaries, and it makes selective calls to the backend for model updates or synchronisation if the user opts in.

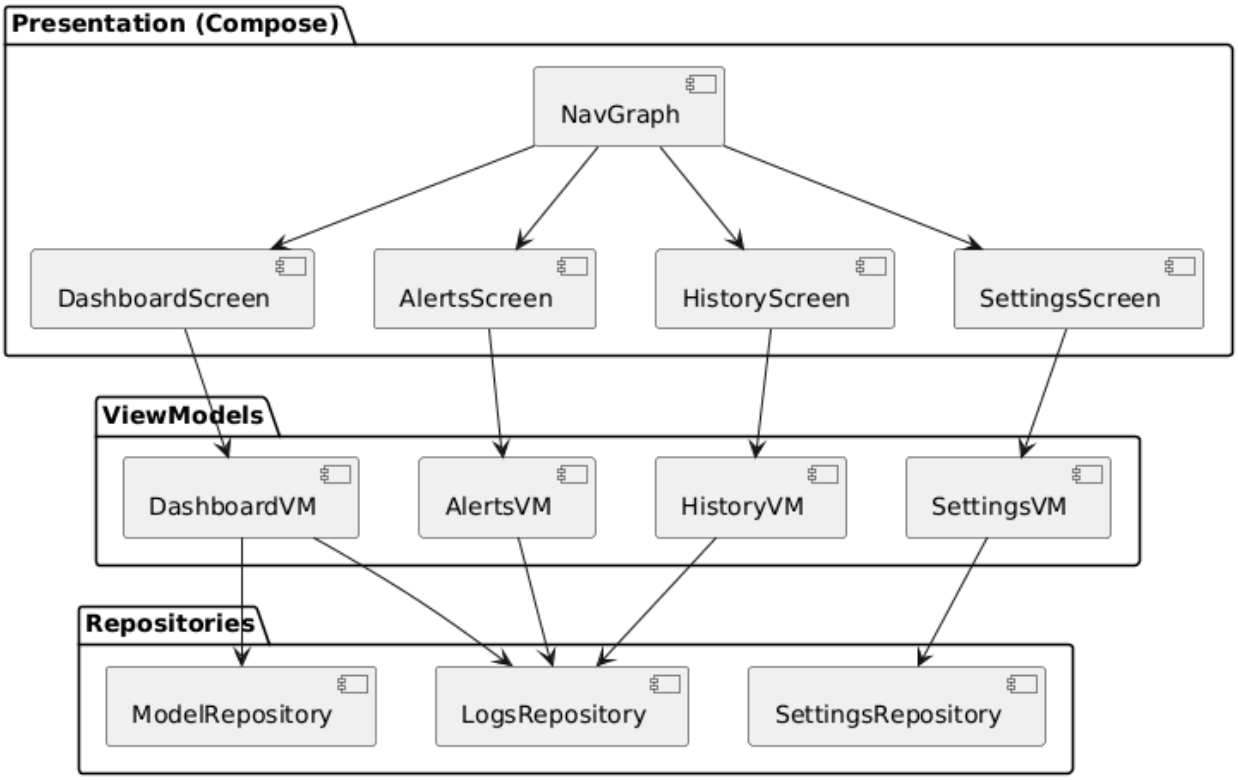
3.1.5 Risks and Mitigations

The two main risks are information overload and reduced performance. Information overload is managed through progressive disclosure: the dashboard highlights key details, while additional records can be explored in the history view. Performance concerns are addressed by using efficient UI update and list rendering techniques to keep the interface responsive even with larger datasets. Accessibility is also taken into account by following established guidelines for colour contrast and supporting adjustable text sizes.

3.1.6 Acceptance and Demonstration Criteria

A working implementation must show the dashboard on launch within the performance budget, display alerts in real time as events are generated, and maintain correct navigation between dashboard, history, and settings even after process death or configuration changes.

Figure 3-1. User Interface Component View — Screens, ViewModels, and Repositories



3.2 Alerts

3.2.1 Introduction

The alerts subsystem converts detection results into actionable messages for the user. Its objective is to deliver notifications that are clear, timely, and useful while avoiding unnecessary disruption.

3.2.2 Structure

Alerts are coordinated through an Alerts ViewModel, which receives detection events from the ONNX Runtime model runner. It evaluates thresholds, applies cooldown policies, and determines the appropriate delivery channel: Android system notifications for background alerts or in-app banners and dialogs for foreground alerts. All alert events are written to local storage for later review.

3.2.3 Flow of Operation

When the model runner produces a suspicious classification, the Alerts ViewModel packages the event with severity, timestamp, and recommended action. If the application is backgrounded, a system notification is triggered through the Android Notification Manager; if foregrounded, a banner or dialog appears in the interface. User responses, such as dismissing or tapping for details, are logged alongside the alert for persistence and traceability.

3.2.4 Interfaces

The alerts subsystem consumes inputs from the model runner and data layer, interacts with Room for persistence, and uses Android's Notification Manager APIs to post system-level notifications [19]. It also interfaces with the Compose UI to render in-app alerts when the application is active.

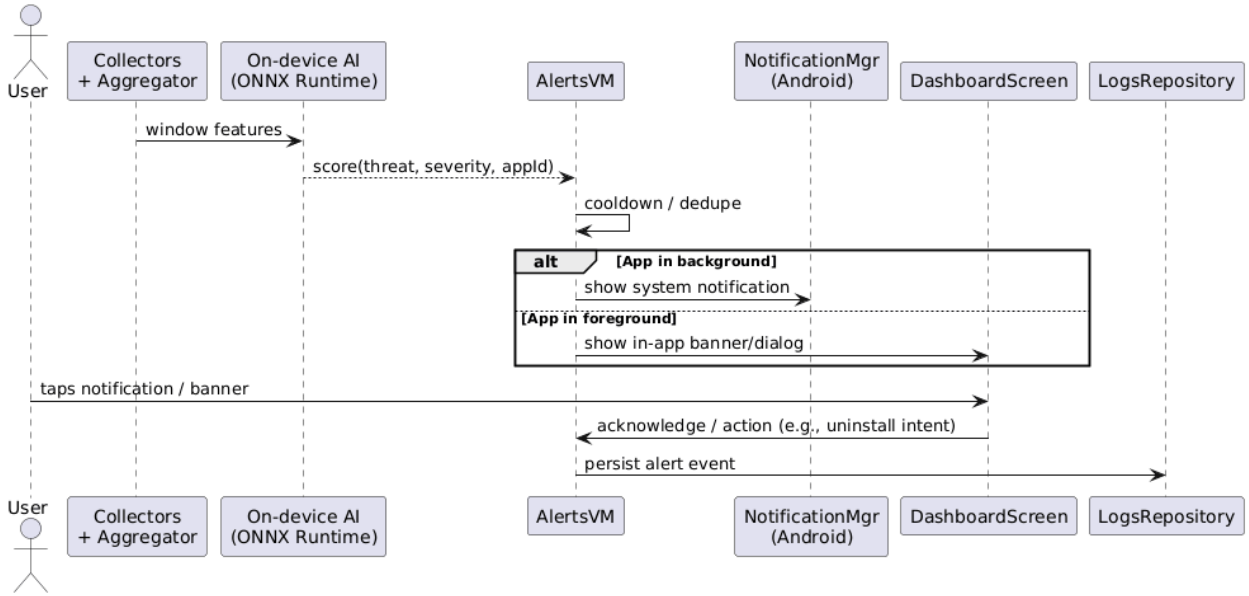
3.2.5 Risks and Mitigations

False positives and over-notification can erode user trust. Mitigation strategies include severity-based alerting, enforced cooldown intervals, and configurable preferences so users can tailor notification behaviour. Recommendations included in alerts (such as "review permissions" or "uninstall suspicious app") ensure that alerts remain actionable rather than merely informational.

3.2.6 Acceptance and Demonstration Criteria

During demonstration, a simulated suspicious event must trigger a notification within one second of detection. The alert should appear in history even after reboot and should dismiss consistently in foreground mode without disrupting other UI flows.

Figure 3-2. Alerts Flow Sequence – Detection to Notification to User Action



4. On-Device Architectural Layer – Data

4.1 Data Collection

4.1.1 Introduction

The purpose of the Data Collection components is to gather OS maintained metadata, this behavioural/static data is what will be used to make comparisons as to determine whether activity is deemed as suspicious or not. The type of data collected is things such as app usage history, app traffic and battery usage. This is to maintain user privacy, this is then converted into features used by the ML model.

4.1.4 Performance and Scheduling

Collections are scheduled with the work managers every couple of minutes, constraints are used to ensure that accurate and consistent readings are made, If the battery of the device it will avoid running. Jobs are purposely kept quite short, being just under a second and batched together to reduce wakeups. Cadence can increase if charging.

4.1.5 Pre-Processing and Feature Windows

The feature vectors will be versioned this way the model and app will stay compatible across ongoing updates. Every window will get raw counters and convert them into fixed vectors taking activity flags and snapshots into burst scores, rates, averages etc to be compared against the app's baseline.

4.1.6 Data Contracts

There are two data contracts which are used. CounterSnapshot, which contains a snapshot of the raw read data. WindowFeatures holds the converted features for the model. When permitted both are serialised to JSON to sync the backend.

4.1.7 Error Paths and Fallbacks

If app usage access is missing, the app will continue in limited state, using the network counters it can read. In cases the counter APIs fail they will be retried with exponential backoff. Any background restriction will be dealt with by the work manager.

4.1.8 External Interfaces

The collectors will rely on the usage stats manager for app usage windows, they depend on the network stats manager and the traffic stats for traffic deltas. Idle state and battery are optionally sampled via the battery manager, this is to make decisions about cadence.

4.1.9 Acceptance and Demo Criteria

Acceptance is met when the model runs on Android 11 or greater without admin access and can produce window figures at the configured cadence, limited state mode still works without usage access and meets the CPU and battery budgets.

4.1.10 Risks and Mitigation

There are three main risks involved: power consumption, OS fragmentation and noisy features. In order to mitigate these we will keep jobs brief, aiming for sub second, then by batching operations together and through adaptive cadence. Smooth windowing and cooldowns along with guarding API calls will combat fragmentation and noisy features.

4.1.11 Traceability

This maps to the data collection and performance constraints outlined in the SRS, also aligning with our decision to rely on OS counters as specified in the TIS

4.2 Permissions Policy

4.2.1 Introduction

The permission policy will follow least privilege, only getting access to what is required. The app will not root the device meaning it does not capture content only OS counters.

4.2.2 Required Access and Justification

The app requires usage access so it is able to read app usage history for the feature windows. Post notifications will deliver real time alerts on Android 13 and above. No dangerous network permissions will be required, the networks stats manager and traffic stats will be enough. A foreground service is used only for user initiated scans.

4.2.3 Permission UX Flows

Before start a pre permission screen will appear explaining to the user what the model does and uses and why. In the case the user decides to decline the app can still run in the limited mode. Battery optimisation guidance will appear only if its impacting function

4.2.4 Privacy and Data Handling

Only metadata will be collected, no payloads or any personal content and it will all be processed on device. Local logs will be stored in Room and will be encrypted. Any optional cloud syncs will be done with encrypted transport.

4.2.5 OS Constraints

The app sticks to background execution by scheduling collection with the work manager under reasonable constraints. Will also avoid using timers which would affect Doze, the foreground service is only started for such user actions.

4.2.6 Risks and Mitigations

To decrease the perceived risks by the user, the app will explain permissions before starting whilst only requesting access for what is necessary. A privacy policy will always be available. Periodically permissions will be audited each release to prevent accidental creep

4.2.7 Logging and Auditing

The app will log permission state transitions, all grants, denials and revocations to Room. These records entries will not include any personal data and will be available to be exported to recreated issues.

4.2.8 Acceptance and Demo Criteria

First demos should clearly show the full flow of the app where permissions are explained on start up and still available to run with full function in limited mode if permissions are denied. Notification prompt will be shown and alerts will arrive as normal on Android 13 or greater, all whilst background collection continues.

4.2.9 Traceability

This maps to the permission and security constraints outlined in the SRS, also aligning with our decision to rely on OS counters under a least privilege model for user trust and safety as specified in the TIS.

4.3 Local Storage

4.3.1 Introduction

The application relies on two main types of local storage to function effectively, structured storage and file-based storage. Structured storage is handled by an Android Room database, an abstraction layer over SQLite, which manages scan results, logs, and configuration data in a reliable and queryable format [5]. File-based storage is used for larger assets, such as the AI model binaries, which are kept in the applications private internal storage directory. Together, these two layers allow the app to retain important information locally, operate without network access, and stay in sync with backend updates when online.

4.3.2 Interfaces

DAOs (Data Access Object)

- ScanResultDao
  For saving and retrieving malware scan results.
- TelemetryDao
  For storing device or event logs.
- ModelMetadataDao
  Records which AI model version is currently in use.
- PendingUploadDao
  Queues results to send to the backend once the device is back online.
- SettingsDao
  Holds user preferences and configuration values.

4.3.3 Operation

When the app runs a malware scan, the result is stored in ScanResult. If the device is offline, the result is also placed in PendingUpload, which a background worker will check later when there is internet access. Model updates are logged in ModelMetadata to make sure the correct version is being used for detection. To stop the database from growing too large, old results are cleared out after a set period.

4.3.4 Relationship to Backend

While Room handles all structured local persistence, it also works in tandem with the backend for specific data. Scan results are written locally first, and if the device is offline they are queued

for later upload. The data uploaded is specifically data used to train the AI model further, to increase its accuracy.
Similarly, the backend can also notify the application of new AI model versions, to ensure the right model is being used during detection. New versions will be sent from the backend to the application's internal storage directory, where it replaces the previous version.

4.3.5 Model Handling in Local Storage

Unlike the applications structured data, the AI model files are kept in the internal storage directory. This directory is private as Android applications are sandboxed by default and persists across reboots. Room itself is not suitable for larger, unstructured assets like the AI model, however it works seamlessly with the internal storage to form the applications local storage solution.

4.3.6 Security and Privacy

The database is encrypted using SQLCipher and the key is managed through the Android Keystore. This ensures that even if someone gains access to the file system, the data is not readable. Sensitive information is kept to a minimum, only data needed for detection and analysis is stored locally, and logs are removed after the retention period [6].

4.3.7 Error Handling

If something goes wrong, such as database corruption, the app falls back to rebuilding the database and starting fresh. Errors during reads or writes are caught and reported through the app’s logging system so they can be reviewed later.

4.3.8 Performance

All database work is done off the main thread to keep the UI responsive. Indexes are applied on the most common query fields, like timestamp and scan status. For bulk inserts, operations are grouped together in transactions to avoid unnecessary overhead.

4.4 Feature Extraction

4.4.1 Overview

The Feature Extraction component is the middle stage of the detection pipeline, positioned between Data Collection and the Model Runner. It consumes CounterSnapshot objects produced by Data Collection and aggregates them into fixed-length windows. From these windows, the component constructs structured, fixed-length feature vectors that represent network activity, application context, and system state. These vectors are the standardised inputs required by the machine learning model to determine whether suspicious data exfiltration behaviour is present.

Because the selected model (XGBoost) is tree-based, it does not require continuous features to be scaled [16]. However, feature extraction is still essential to:

- aggregate raw counters into meaningful metrics,
- encode categorical values (e.g. network type, screen state),
- enforce a consistent schema and feature order, and
- handle unavailable telemetry by marking them with a missingMask, ensuring the model can still run in limited mode.

By enforcing consistent ordering, versioning, and encoding, this component ensures that all downstream processing, including storage in the encrypted repository and inference via ONNX Runtime Mobile, remains reliable and compatible across devices.

4.4.2 Structural Details

The Feature Extraction component is structured as a pipeline of three internal subcomponents. Each subcomponent has a distinct role, ensuring modularity, maintainability, and testability across the detection system.

1. **Window Manager (Consumer of Data Collection)**

- Consumes CounterSnapshot objects produced by the Data Collection service.
- Groups raw telemetry into fixed-length windows (default: 60 s, sliding every 30 s).
- Handles late or missing data with a short tolerance to preserve consistency across windows.
- Emits Window { windowId, startEpoch, endEpoch, samples[] } objects to the Feature Extractor.

2. **Feature Extractor (Core Transformer)**

- Aggregates raw counters within each window (e.g., total bytes sent, number of foreground app switches, battery percentage change).

- Encodes categorical/contextual features into numerical form (e.g. netType as one-hot, screenState as binary).

- Produces a fixed-length feature vector aligned with the model's schema, enforced via ModelMeta.

- Adds a missingMask to mark unavailable inputs (e.g. denied Usage Access permission). This enables the model to run in a "limited mode" rather than failing completely.

- Because XGBoost is a tree-based model, explicit feature scaling is not required.

3. **Feature Repository (Producer of Outputs)**

- Persists feature vectors (WindowFeatures) in an encrypted local database using Room and SQLCipher.

- Maintains schema and feature versioning (featuresVersion, schemaVersion) to ensure compatibility with the deployed model.

- Exposes APIs for downstream components:

    - The Model Runner queries stored vectors for inference.

    - The optional Upload Worker serialises vectors into JSON and sends them via authenticated APIs (JWT/OAuth2).

This structure positions the Feature Extraction component firmly between Data Collection and the Model Runner. Upstream, it consumes raw telemetry snapshots. Downstream, it produces consistent, privacy-preserving feature vectors suitable for inference or secure upload. By separating windowing, extraction, and persistence, the design ensures that failures in one stage (e.g. missing counters) do not break the entire detection pipeline.

4.4.3 Interfaces

The Feature Extraction and Selection component forms the bridge between raw telemetry and the machine learning model.

On the input side, it interfaces with the Data Collection component. Data Collection delivers Window objects, which represent short, fixed periods of telemetry (60 s). Each window contains multiple CounterSnapshot samples, including fields such as bytes sent and received, screen state, battery level, and network type. These windows provide the raw ingredients required for further processing.

The Feature Extractor aggregates these values into a WindowFeatures object. A WindowFeatures object contains a fixed-length vector, where each index always represents the

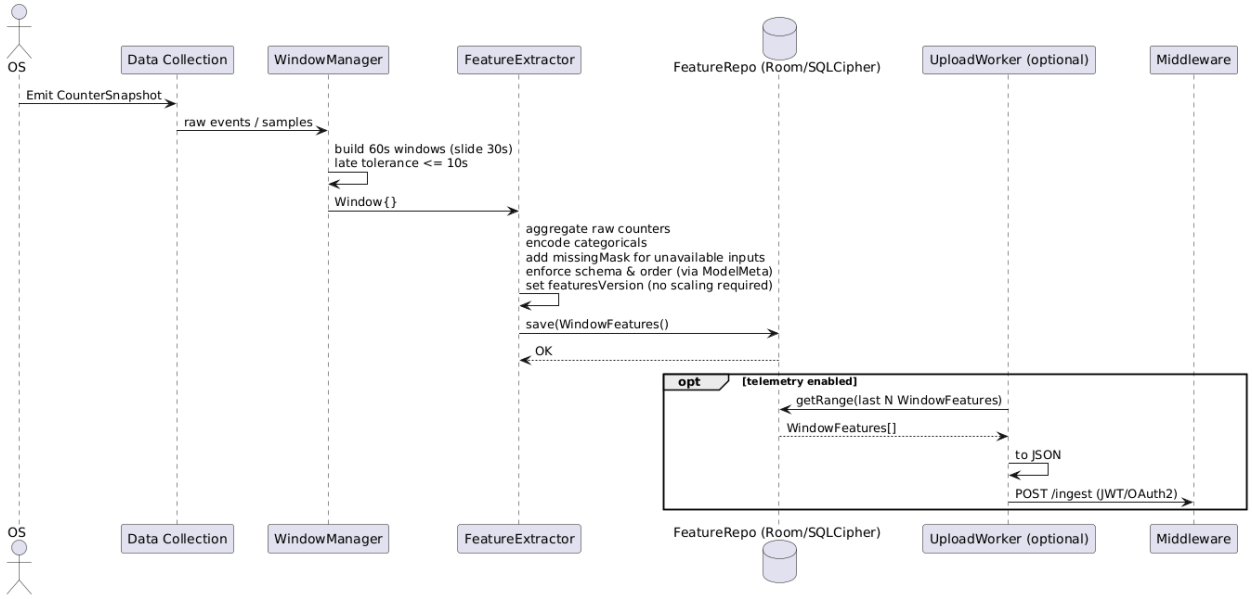
same type of information (total bytes out, battery change, or binary screen state). This consistent format is essential, as machine learning models expect input vectors to follow a fixed schema. If some data sources are unavailable (e.g. the user has denied Usage Access permission), those indices are marked in a missingMask, which allows the model to operate in a limited mode without breaking.

On the output side, the feature vectors are written into the encrypted local database using Room and SQLCipher. The Model Runner then retrieves these stored vectors, converts them into tensors, and passes them to the ONNX Runtime Mobile engine for on-device inference. The inference step produces a probability score indicating suspicious or normal behaviour.

If telemetry sharing is enabled, the same feature vectors can also be serialised into lightweight JSON and uploaded by the Upload Worker. These uploads are designed to be privacy-preserving, excluding sensitive details such as app names or IP addresses. All outbound communication is authenticated using JSON Web Tokens (JWT), a widely adopted standard for secure API access.

In this way, the Feature Extraction interfaces ensure that operating system counters are consistently transformed into usable model inputs, while maintaining privacy, enabling schema versioning, and supporting inference through ONNX Runtime Mobile.

4.4.3 Sequence Diagram



4.5 Model Runner

4.5.1 Overview

The Model Runner is the final stage of the on-device detection pipeline. It consumes the standardised WindowFeatures objects produced by the Feature Extraction component and evaluates them using a trained machine learning model deployed with ONNX Runtime Mobile.

Before executing inference, the Model Runner validates that incoming vectors conform to the expected schema and feature version. Once validated, the vectors are converted into tensors and passed into the runtime for scoring. The inference produces a probability score (0.0–1.0) representing the likelihood of suspicious behaviour. This score is then thresholded and transformed into a structured InferenceResult containing the window identifier, probability score, and final classification label (“BENIGN” or “SUSPICIOUS”).

By performing inference locally on-device, the Model Runner ensures:

- Privacy preservation, since no raw feature vectors or scores leave the device by default.
- Low latency, enabling real-time responsiveness for alerting.
- Offline resilience, avoiding reliance on network connectivity for threat detection.

This design mirrors the responsibilities of Feature Extraction – enforcing schema consistency, versioning, and modularity – while focusing on the final classification step of the pipeline.

4.5.2 Structural Details

The Model Runner is composed of three internal subcomponents, each responsible for a distinct concern:

1. Input Validator

- Confirms that incoming WindowFeatures match the model’s expected schema (length, feature order, data type).
- Verifies feature version compatibility (e.g. featuresVersion ≥ minFeaturesVersion stored in ModelMeta).
- Rejects incompatible vectors and emits clear error states such as SCHEMA_MISMATCH or VERSION_INCOMPATIBLE, preventing undefined model behaviour.

2. ONNX Inference Engine

- Loads the deployed ONNX model packaged with the application.
- Uses ONNX Runtime Mobile for efficient inference on Android devices, leveraging hardware acceleration (CPU, GPU, or NPU) where available.
- Converts the validated feature vector into a tensor (shape [1, N]), executes the model, and extracts the raw probability score.
- Benefits from ONNX's optimisation features such as quantisation and kernel caching to reduce runtime latency on mobile platforms.

3. Result Generator

- Applies a configurable decision threshold (≥0.80 == suspicious).
- Packages the outcome into a standardised InferenceResult object containing:
    - windowId – reference to the originating telemetry window.
    - score – probability of suspicious behaviour (0.0–1.0).
    - label – final classification ("BENIGN" or "SUSPICIOUS").
- Provides results to downstream consumers:
    - Alerts UI for real-time notification to the user.
    - Telemetry queue (optional) for aggregated, privacy-preserving research uploads.

This separation of validation, inference, and result generation ensures that failures in one step (e.g. invalid schema) do not cascade into downstream processing, maintaining robustness across devices.

4.5.3 Interfaces

The Model Runner sits directly downstream of the Feature Extraction and Selection component and upstream of the UI/Alerting and optional Telemetry pipeline. Its interfaces are:

Input Interface

- Consumes WindowFeatures objects retrieved from the encrypted Feature Repository (Room + SQLCipher).

- Each object contains:
    - vector[] — fixed-length numeric feature vector.
    - missingMask[] — markers for unavailable inputs.
    - featuresVersion and schemaVersion — for compatibility checks.
- The Input Validator confirms these values before inference.

Internal Interface

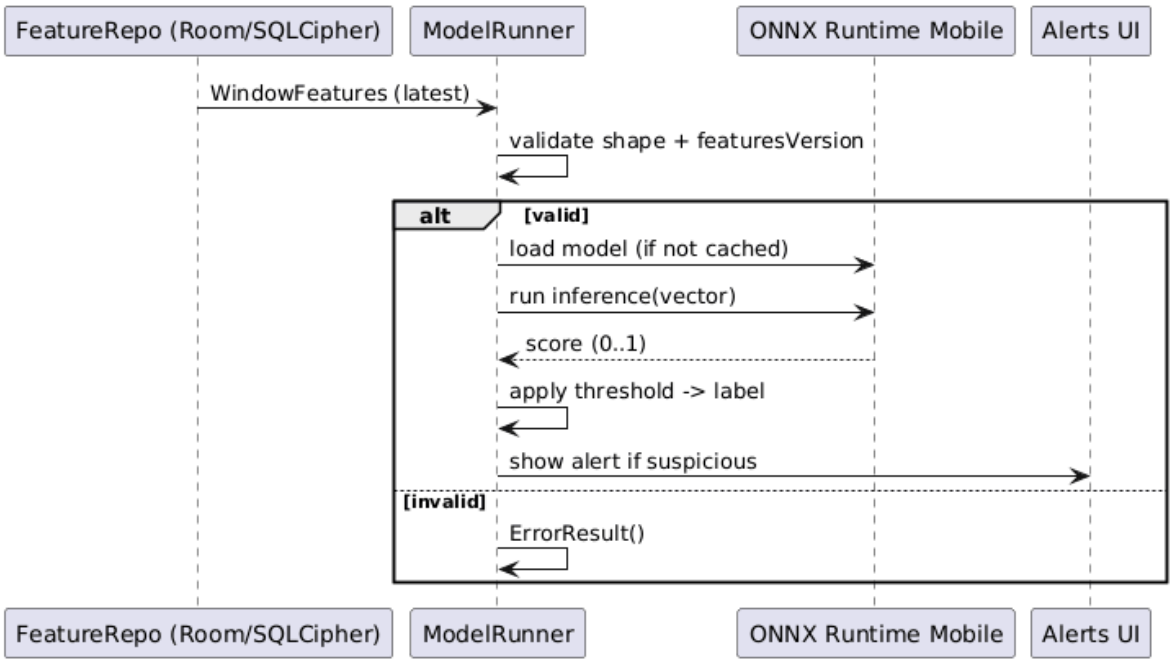
- Passes the validated feature vector into the ONNX Runtime Mobile session as a tensor.
- This interaction is isolated inside the Model Runner, remaining transparent to upstream and downstream components.

Output Interface

- Produces an InferenceResult object containing:
    - windowId (traceability to telemetry window).
    - score (probability between 0.0–1.0).
    - label (“BENIGN” or “SUSPICIOUS”).
- Results are consumed by:
    - Alerts UI, which displays real-time notifications to the user.
    - Upload Worker (if telemetry sharing is enabled), which serialises outcomes into privacy-preserving JSON for backend ingestion.

This interface design keeps the Model Runner loosely coupled, depending only on standardised inputs and producing a single, consistent output object.

4.5.4 Sequence Diagram



5. Off-Device Architectural Layer – Online Storage

5.1 Introduction

This chapter defines the server-side architecture that supports the mobile malware-detection client. The service is implemented in Spring Boot [8] and exposes a small, versioned REST API that distributes configuration and model artifacts to devices and optionally ingests device telemetry. The chapter describes the components of the service, how they collaborate at runtime, the external interfaces and data formats, and the security, diagnostics, and deployment characteristics required to operate the system reliably [8][9].

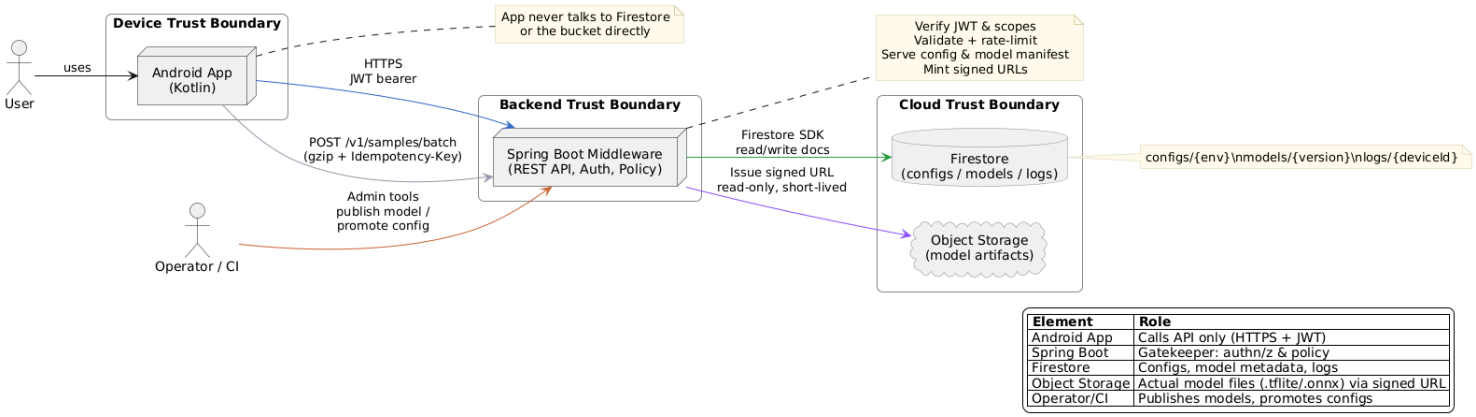


Figure 5-1 – System Context (Online Storage Layer)

5.2 System Structural Overview

We are keeping the back-end storage simple and predictable: the Android app never talks to the database directly. It only talks to our Spring Boot middleware over HTTPS [8]. The middleware will read and write Firestore documents on behalf of the app [11] and hand out short-lived signed URLs for the actual AI model files that live in object storage [12]. This gives us one trusted place to enforce policy (auth, validation, rate limits) and one tidy source of truth for the documents the app depends on (configs, model metadata, and telemetry logs).

5.2.1 Software Component View

At the storage level there are three document areas each with a clear job:

- Configs store (configs/{env}): one compact JSON per environment (e.g., dev, stage, prod) that holds the live settings; detection threshold, feature flags, and minModelVersion. The middleware reads this to serve GET /v1/config. End-user devices never write here; only the service account (or an operator tool) promotes changes.

- Models store (models/{version}): one document per released model carrying version, sha256 checksum, size, format (onnx) [14], artifactPath (where the file is in the bucket), minAppVersion. The middleware reads this to answer GET /v1/models/latest, adds an ETag header [13], and returns a signed URL [12] so the device can download the artifact directly from the bucket and verify the checksum before activating it.

- Logs store (logs/{deviceId}/yymm/{autoId}): append-only records of detections or lightweight telemetry the app chooses to upload in batches. The middleware validates and batch-writes these after POST /v1/samples/batch; devices can't see other devices' logs.

The actual model files will not live in Firestore; they will live in an object storage bucket under a simple path like models/{version}.{onnx} [14]. We will never stream those big files through the API; issuing time-boxed signed URLs is cheaper, faster, and safer [12].

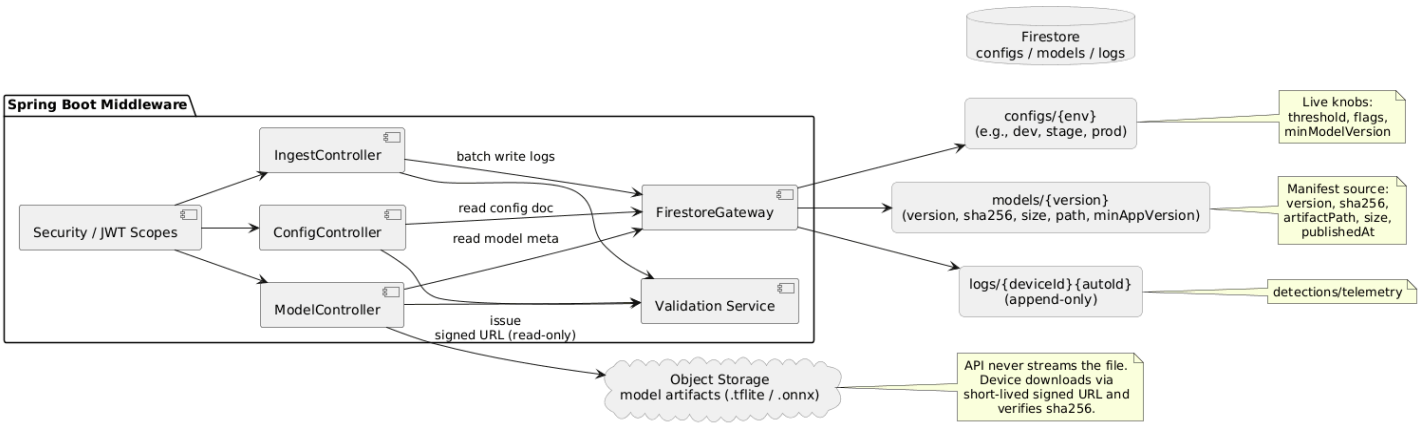


Figure 5-2 – Storage Component View (Online Storage Layer)

5.2.2 Data Organisation and Access Paths

Read/write paths are straightforward:

- Config fetch: Middleware reads configs/{env} and returns a tiny JSON to the app. The app caches it with a TTL and adapts behaviour immediately (no APK update required).

- Model discovery: Middleware reads models/{version} and, if the device's If-None-Match doesn't match our latest, returns the manifest plus a signed URL. The device downloads from the bucket, checks the sha256, then swaps models.

- Telemetry ingest: The app posts a gzipped batch; middleware authenticates, rate-limits, validates shape and size (e.g., ≤128 KB, ≤100 events), and batch-writes under logs/{deviceId}/. The app marks successfully persisted items as "synced."

5.3 Theory of Operation

The app never directly communicates with back-end storage. It always talks to the middleware, and the middleware accesses Firestore and the bucket. Here's how the moving parts run at runtime.

5.3.1 Config fetch (live knobs)

On app start (and then on a timer), the app calls GET /v1/config with its JWT [9][10]. The middleware verifies the token, reads configs/{env} from Firestore, and returns a tiny JSON with the threshold, feature flags, and minModelVersion. The app caches this locally with a TTL (e.g., 6-12 hours). If the device is offline, it uses the last cached config and logs that it's running on stale policy. When the config changes (e.g., we raise the threshold), devices pick it up automatically on their next refresh - no APK update required.

5.3.2 Model discovery, download, and activation

After fetching config (or when minModelVersion increases), the app checks whether it needs a newer model. It calls GET /v1/models/latest and includes If-None-Match with its current model version [13]. The middleware compares versions by reading models/{version}. If nothing changed, it returns 304 Not Modified and the phone keeps its model. If there's a new version, it returns 200 with a model manifest: {version, sha256, size, format, artifactUrl, minAppVersion, publishedAt} and sets ETag: "<version>" [13].

The app then downloads the artifact directly from the bucket using the signed URL (short-lived, least-privilege) [12]. It verifies the sha256 from the manifest before activating. Keep the previous model on disk as a backup. If checksum verification fails or storage is full, the app discards the download, keeps the old model, and reports an error; the middleware logs this (via telemetry if enabled).

5.3.3 Telemetry ingest

The app groups detections/telemetry into a gzipped batch and posts it to POST /v1/samples/batch with an Idempotency-Key. The middleware authenticates, applies per-device rate limits, validates schema/size (eg. ≤ 128 KB gz; ≤ 100 events), and then batch-writes to logs/ via the Firestore gateway. On success it returns 202 Accepted. If some items fail validation, it returns 207 Multi-Status with per-item reasons; the client marks successes as “synced” and only retries the failures with exponential backoff. If the network is down, WorkManager queues the batch and retries later; idempotency ensures we don’t duplicate on retries [15].

Consistency, caching, and promotion

- Configs are eventually consistent across devices: we write configs/dev first for testing, then promote to configs/stage, and finally to configs/prod. Devices in “prod” only read configs/prod.
- Manifests are the source of truth for models; the bucket is just a file host. The app trusts the manifest’s sha256, not the file name.
- ETag prevents wasteful downloads, and the app’s local cache avoids re-reading config every minute [13].

Failure modes and expected behaviour

- Authentication fails (expired/invalid JWT): middleware returns 401/403; app refreshes the token and retries [9][10].
- Rate limit hit: middleware returns 429 with Retry-After; app backs off.
- Payload too large / bad schema: 413/400 with a clear error code; app trims the batch or fixes fields before retrying.
- Firestore unavailable: middleware returns 5xx; app keeps data locally and retries later (uploads stay idempotent).
- Checksum mismatch on model: app discards the file, keeps the old model, and tries again on the next cycle.

House rules the runtime relies on

- The app never writes configs/* or models/*; only the service account does.
- Logs are append-only and scoped to the device (logs/{deviceId}/…).
- Every request carries or returns a correlation ID (the X-Request-Id) so a single run can be traced from phone → middleware → storage in logs and metrics.

5.4 List of External Interfaces

Interface	Path / Namespace	Direction	Who Calls	Payload / Shape	Notes
Config read	configs/{env} e.g. configs/prod	Read	Middleware	{ threshold:number, minModelVersion:string, flags:map, updatedAt, updatedBy }	Feeds GET /v1/config; devices cache with TTL
Config write (promotion)	configs/{env}	Write	Operator tool / middleware SA	Same as above	Dev → Stage → Prod promotion flow
Model metadata read	models/{version}	Read	Middleware	{ version, format, sha256, size, artifactPath, minAppVersion, publishedAt }	Feeds GET /v1/models/latest (adds ETag)
Model metadata write (publish)	models/{version}	Write	Operator tool / middleware SA	Same as above	Written after artifact upload & hash calc
Log batch write (optional)	logs/{deviceId}/yymm/{autoId}	Write	Middleware	{ ts, appId, features{}, score?, label?, modelVersion, serverTimestamp }	Created from POST /v1/samples/batch
Log query (ops only)	logs/{deviceId}/yymm/*	Read	Analyst/ops tools	Query by (deviceId, ts) window	Devices cannot read other devices' logs

5.5 Security and Access Control

We enforce identity with JWT, authorisation with scopes, storage policy with Firestore rules, and model access with short-lived signed URLs.

- Device app: authenticates to the middleware with a JWT. Middleware verifies issuer/audience/signature/expiry and checks scopes before doing anything [9][10].

- Middleware service account: the only principal allowed to write configs/*, models/*, and to write logs/* on behalf of devices; also the only principal that can mint signed URLs for the model bucket [11][12].

- Operator / CI: uses an admin tool (behind auth) to publish models and promote configs; writes are audited (updatedBy, updatedAt) and executed with the service account.

Fail-closed: missing/invalid JWT throws error 401; valid JWT without scope throws 403. Middleware never attempts a Firestore write if authentication fails.

5.6 Product Deliverables (Storage)

5.6.1 Programmatic Interfaces (APIs)

Component Name	Location (repo path)	Brief Description
OpenAPI spec (openapi.yaml)	api/openapi.yaml	Defines the storage-facing API the app uses via middleware: GET /v1/config, GET /v1/models/latest (ETag, signed URL), and optional POST /v1/samples/batch (idempotent, gzipped).
GET /v1/config	Implemented in server/src/.../ConfigController	Returns config doc from configs/{env}; tiny JSON with threshold, minModelVersion, and flags.
GET /v1/models/latest	Implemented in server/src/.../ModelController	Returns model manifest from models/{version} with ETag and a short-lived signed URL to the artifact in the bucket.
POST /v1/samples/batch	Implemented in server/src/.../IngestController	Validates, rate-limits, and batch-writes logs under logs/{deviceId}; uses Idempotency-Key to prevent duplicates.

5.7 Debugging, Error Logging, and Performance Monitoring

5.7.1 Observability & Logging

Every request gets an X-Request-Id at the middleware, and we carry that same ID into Firestore reads/writes and signed-URL issuance. Logs are short, structured JSON and PII-free (no tokens or signed URLs; device IDs hashed/truncated). The error body the app sees is always the same: { code, message, correlationId }, so client handling is uniform and we can jump straight to the matching server log line.

5.7.2 Metrics & Dashboards

We track the basics that matter: route-level latency and error rates for /v1/config, /v1/models/latest, and /v1/samples/batch; ingest volume/quality (accepted vs partial vs rejected) and rough batch size; Firestore read/write timing; and a count of signed URLs issued (quick signal during model rollouts). One simple dashboard shows API health, one shows ingest, and one shows storage. Alerts are light: slow config/batch endpoints, high reject rates, or stalled rollouts (apps polling but no signed URLs issued).

5.7.3 Troubleshooting & Retention

Start with the correlation ID: search it to see the full path phone, middleware, storage. For “config not updating,” check auth then the configs/prod doc; for “model keeps downloading,” verify ETag and checksum; for “uploads failing,” look for rate-limit hits or size/schema errors, then back off or trim. Logs in Firestore have a defined TTL/archival window; config and model metadata are kept indefinitely for audit.

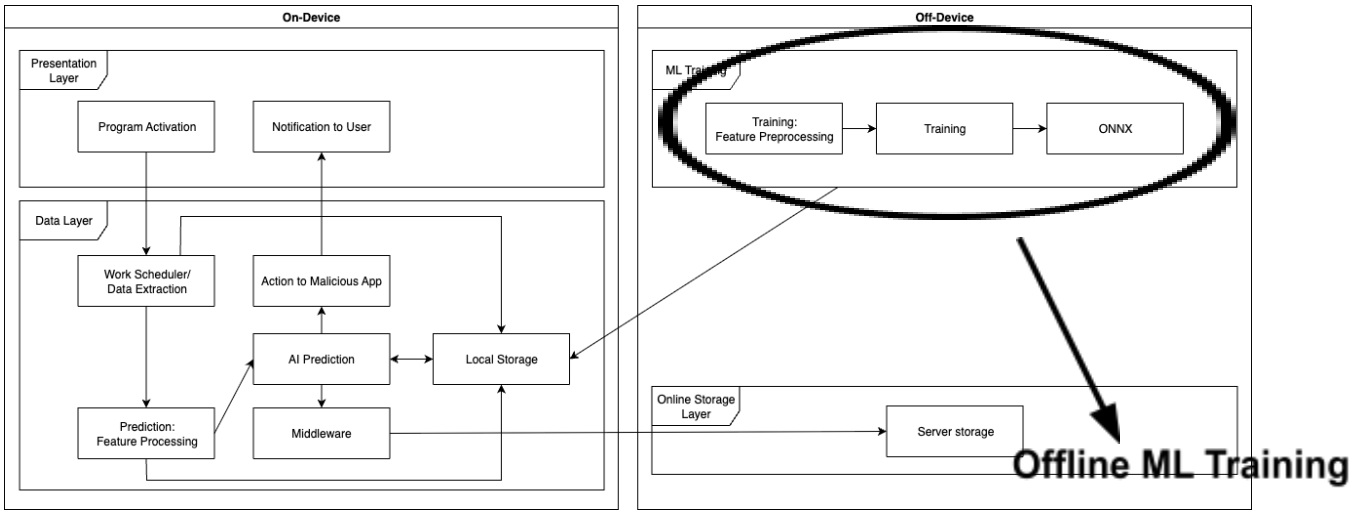
6. Off-Device Architectural Layer – Offline ML Training

6.1 Introduction

This section refers to the offline creation of the AI model for malware detection that will be deployed in the app. The offline ML training process occurs off-device in a controlled environment and later deployed in the app to enable real-time and offline malware detection. The goals of the offline ML training process is to create a model that is accurate in distinguishing between benign and malicious applications, lightweight so that it can operate with minimal resources on a mobile device that typically offers limited resources, and that is compatible with the Local Storage component on-device.

6.2 Structure Explanation

6.2.1 View of Component



The offline ML training creates an AI model that is trained on examples of benign and malicious activity with a combination of features and is converted into a version suitable for mobile devices.

6.2.2 Relationship to Local Storage Component

6.2.2.1 On-Device Model Integration

The model created from the ML training is used to make predictions based on the data of an application that is extracted in the Data Extraction component. The training data that the model is trained on will include the same features as the extracted data in the Data Extraction component.

The exported model is packaged with the application so the model will be downloaded along with the application in the Local Storage component. At runtime the Local Storage component will load the model locally and output its prediction score on how likely the scanned application is malicious.

To add the model to the device the dependency *com.microsoft.onnxruntime:onnxruntime-android:<latest-release>* is added to the application's *app:build.gradle.kts / app:build.gradle*. Then a *raw* resource file is created in the Android project and the generated *linear_model.onnx* file is pasted into it [4].

6.2.2.2 ONNX ai.onnxruntime Interface

6.2.2.2.1 What It Does

The ai.onnxruntime interface works by providing a runtime environment for the trained model to receive data and create predictions on the application in multiple environments, which in this case would be the Android environment on mobile devices.

6.2.2.2.2 ai.onnxruntime.OrtEnvironment

OrtEnvironment is an object that represents the runtime environment for ONNX and manages the global state and resources that are required for the running models [4]. It initialises the ONNX Runtime on the device making sure the system resources such as memory and execution providers are properly allocated.

6.2.2.2.3 ai.onnxruntime.OrtSession

OrtSession represents the running session for the model which includes methods to provide inferences [4]. It loads the ONNX model into memory and executes it against input data provided by the application.

6.3 How It Works

6.3.1 Feature Preprocessing

The training dataset includes labeled examples of both benign and malicious application behaviour. Feature selection focuses on relevant, discriminative attributes such as API call

sequences, permission usage, and behavioural patterns. Training data is processed in which relevant features are kept, however, since XGBoost doesn't require particular preprocessing techniques such as feature scaling [1], all that is required is removing incorrect data and unnecessary data.

6.3.2 Model Training

The model used is XGBoost on Python as it is known to have high performance on tabular and categorical datasets relevant to those needed for malware detection. The training phase consists of iteratively building an ensemble of decision trees which optimises classification accuracy between benign and malicious test samples. Furthermore, through cross-validation: RandomizedSearchCV(), the hyperparameters are tuned to minimise overfitting and increase generalisation. [3]

6.3.3 ONNX

Once the model is finished training, it is then converted to a more compatible format to use within Android devices. The model is converted to ONNX format to standardise the model and enable interoperability for use on Android. Furthermore, the model can be optimised for use on mobile devices through techniques such as quantisation where it converts the model to reduce the weights and activations for a smaller, faster and more efficient model [2].

6.4 Performance Monitoring

The performance will be monitored to optimise the XGBoost model by using a validation split of 80% training and 20% testing and creating an AUC score to find the accuracy of the model. This AUC score will provide how the model is performing to its hyperparameter tuning and the quality of its training data.

7. References

[1] G. Jha, "Feature scaling in machine learning: Which popular algorithms require it and which don't?" Medium, Oct. 1, 2024. https://medium.com/@post.gourang/feature-scaling-in-machine-learning-which-popular-algorithms-require-it-and-which-dont-a71f5585d664

[2] Microsoft, "Deploy on mobile | onnxruntime," ONNX Runtime Documentation. https://onnxruntime.ai/docs/tutorials/mobile/

[3] N. Apichardsilkij, "Basic Comparison Between RandomForest, SVM, and XGBoost," *Medium*, Mar. 06, 2024.

https://medium.com/@ap.nattapoj_st/basic-comparison-between-randomforest-svm-and-xgboost-0e5862871175

[4] V. Kumar, “Deploying common machine learning & deep learning models on Android and iOS using ONNX (Open Neural Network Exchange),” Medium, Mar. 30, 2020. https://medium.com/@vinodkumarg/deploying-common-machine-learning-deep-learning-models-on-android-and-ios-using-onnx-open-neural-e30cb118b498

[5] Android Developers, “Save data in a local database using Room,” *Android Developers (Training: Data Storage, Room)*. Available: https://developer.android.com/training/data-storage/room

[6] Zetetic LLC, “About SQLCipher — Information and History,” *Zetetic*, 2025. Available: https://www.zetetic.net/sqlcipher/about/

[8] Spring, *Spring Boot Reference Documentation* (incl. Security & Actuator). https://docs.spring.io/spring-boot/docs/current/reference/html/

[9] IETF, RFC 7519, “JSON Web Token (JWT),” May 2015. https://www.rfc-editor.org/rfc/rfc7519

[10] Google, *Firebase Authentication* (mobile JWT/OAuth2 usage). https://firebase.google.com/docs/auth

[11] Google Cloud, *Cloud Firestore Documentation* (collections, rules, indexes). https://cloud.google.com/firestore/docs

[12] Google Cloud, *Cloud Storage: Signed URLs* (time-boxed object access). https://cloud.google.com/storage/docs/access-control/signed-urls

[13] IETF, RFC 7232, “HTTP/1.1 Conditional Requests” (ETag / If-None-Match). https://www.rfc-editor.org/rfc/rfc7232

[14] ONNX, *Open Neural Network Exchange (ONNX) Format* (model packaging). https://onnx.ai/

[15] Android Developers, *WorkManager* (background retries and scheduling). https://developer.android.com/topic/libraries/architecture/workmanager

[16] “Feature Engineering for XGBoost,” XGBoosting, 2024. Available: https://xgboosting.com/feature-engineering-for-xgboost/

[17] Android Developers. *Jetpack Compose — Modern toolkit for building native UI*. Google. Available at: https://developer.android.com/jetpack/compose

[18] Android Developers. *Navigation in Compose*. Google. Available at: https://developer.android.com/jetpack/compose/navigation

[19] Android Developers. *Notifications Overview*. Google. Available at: https://developer.android.com/develop/ui/views/notifications

8. Group Attribution

Victor Gan: **5**

Tobias Jun: **1, 2.1, 2.2, 2.3, 6**

Atticus Kong: **3**

Mark Pham: **4.4, 4.5**

Jacob Sepkus: **4.3**

Nick Williamson: **4.1, 4.2**